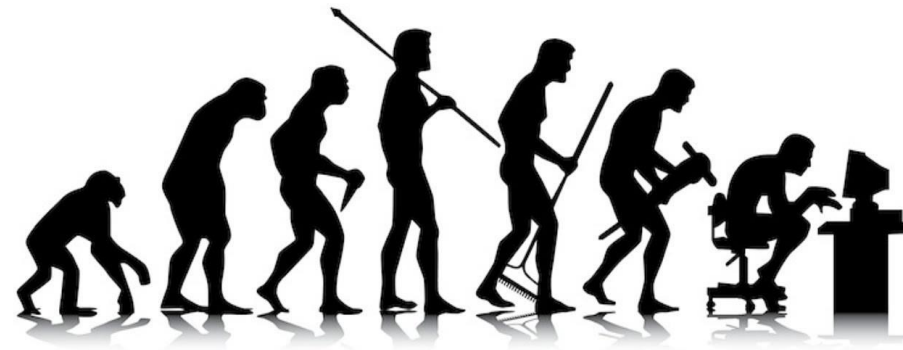


Collections Aufgaben



Object Caster

Schreibe eine Methode, die eine Object-Liste von Zahlen erhält.

Iteriere über die Liste und berechne für jede Zahl das Quadrat.

Gebe die Quadrate als neue Liste zurück.

```
public static List<Integer> toSquaredIntegerList(List<Object> objectList)
```

Falls ein Object nicht gecastet werden kann, soll das Objekt **nicht** in die Ergebnisliste mit aufgenommen werden.

Schreibe einen Unit Test die Klasse.

Object Caster: Lösung

```
public class ObjectCaster {  
  
    public static List<Integer> toSquaredIntegerList(List<Object> objectList) {  
        List<Integer> integerList = new ArrayList<>(objectList.size());  
        for (Object object : objectList) {  
            if (object instanceof Integer) {  
                int integer = (Integer) object;  
                integerList.add(integer * integer);  
            }  
        }  
        return integerList;  
    }  
  
    // Alternative  
    public static List<Integer> toSquaredIntegerListWithPatternMatching(List<Object> objectList) {  
        List<Integer> integerList = new ArrayList<>(objectList.size());  
        for (Object object : objectList) {  
            if (object instanceof Integer integer) { // Pattern Matching ist ein relativ neues Feature  
                integerList.add(integer * integer);  
            }  
        }  
        return integerList;  
    }  
}
```

Object Caster: Lösung

```
@Test
void negativeNumbers() {
    // Arrange
    List<Object> objectList = List.of(1, -3);

    // Act
    List<Integer> integerList = ObjectCaster.toSquaredIntegerList(objectList);

    // Assert
    assertEquals(List.of(1, 9), integerList);
}
```

```
@Test
void mixedTypes() {
    // Arrange
    List<Object> objectList = List.of(1, "test", 2, 3.5, 3);

    // Act
    List<Integer> integerList = ObjectCaster.toSquaredIntegerList(objectList);

    // Assert
    assertEquals(List.of(1, 4, 9), integerList);
}
```

```
@Test
void onlyIntegers() {
    // Arrange
    List<Object> objectList = List.of(2, 3, 4);

    // Act
    List<Integer> integerList = ObjectCaster.toSquaredIntegerList(objectList);

    // Assert
    assertEquals(List.of(4, 9, 16), integerList);
}
```

```
@Test
void emptyList() {
    // Arrange
    List<Object> objectList = Collections.emptyList();

    // Act
    List<Integer> integerList = ObjectCaster.toSquaredIntegerList(objectList);

    // Assert
    assertTrue(integerList.isEmpty());
}
```

```
@Test
void noIntegers() {
    // Arrange
    List<Object> objectList = List.of("a", "b", 3.5, true);

    // Act
    List<Integer> integerList = ObjectCaster.toSquaredIntegerList(objectList);

    // Assert
    assertTrue(integerList.isEmpty());
}
```

```
@Test
void withNull() {
    // Arrange
    List<Object> objectList = new ArrayList<Object>();
    objectList.add(2);
    objectList.add(null);
    objectList.add(3);

    // Act
    List<Integer> integerList = ObjectCaster.toSquaredIntegerList(objectList);

    // Assert
    assertEquals(List.of(4, 9), integerList);
}
```

ListConverter

Schreibe eine Klasse ListConverter und implementiere die folgenden Methoden:

// Wandle die gegebene List<Integer> in ein Integer[]-Array um und gebe das Array zurück.

```
public Integer[] toArray(List<Integer> list)
```

// Wandle das gegebene Integer[]-Array in eine List<Integer> um und gebe die List zurück.

```
public List<Integer> toList(Integer[] array)
```

Schreibe einen Unit Test die Klasse.

ListConverter: Lösung

```
public class ListConverter {

    public Integer[] toArray(List<Integer> list) {
        return list.toArray(Integer[]::new);

// return list.toArray(new Integer[0]);

// Integer[] array = new Integer[list.size()];
// for (int i = 0; i < list.size(); i++) {
//     array[i] = list.get(i);
// }
// return array;
    }

    public List<Integer> toList(Integer[] array) {
        return Arrays.asList(array);

// return List.of(array); // erzeugt eine immutable Liste

// var list = new ArrayList<Integer>(array.length);
// list.addAll(List.of(array));
// return list;

// for (Integer integer : array) {
//     list.add(integer);
// }

// return List.of(array);
    }
}
```

```
class ListConverterTest {
```

```
@Test
void testToListWithNonEmptyArray() {
    // Arrange
    var array = new Integer[]{1, 2, 3};
    var listConverter = new ListConverter();

    // Act
    var result = listConverter.toList(array);

    // Assert
    assertEquals(Arrays.asList(1, 2, 3), result);
}
```

```
@Test
void testToListWithEmptyArray() {
    // Arrange
    var array = new Integer[]{};
    var listConverter = new ListConverter();

    // Act
    var result = listConverter.toList(array);

    // Assert
    assertTrue(result.isEmpty());
}
```

```
@Test
void testToListWithNull() {
    // Arrange
    var array = new Integer[]{1, null, 3};
    var listConverter = new ListConverter();

    // Act
    var result = listConverter.toList(array);

    // Assert
    assertEquals(Arrays.asList(1, null, 3), result);
}
```

```
@Test
void testToArrayWithNonEmptyList() {
    // Arrange
    var list = List.of(1, 2, 3);
    var listConverter = new ListConverter();

    // Act
    var result = listConverter.toArray(list);

    // Assert
    assertEquals(new Integer[]{1, 2, 3}, result);
}
```

```
@Test
void testToArrayWithEmptyList() {
    // Arrange
    List<Integer> list = new ArrayList<>();
    var listConverter = new ListConverter();

    // Act
    var result = listConverter.toArray(list);

    // Assert
    assertEquals(new Integer[] {}, result);
}
```

```
@Test
void testToArrayWithNull() {
    // Arrange
    var list = Arrays.asList(1, null, 3);
    var expected = new Integer[]{1, null, 3};
    var listConverter = new ListConverter();

    // Act
    var actual = listConverter.toArray(list);

    // Assert
    assertEquals(expected, actual);
}
```

MyStack<T>

Implementiere eine generische Klasse MyStack<T>, die **intern** eine **LinkedList** verwendet und folgende Methoden besitzt:

// Fügt das element ans Ende des Stacks hinzu.

```
public void push(T element)
```

// Entfernt das zuletzt gepushte element aus dem Stack und gibt es zurück.

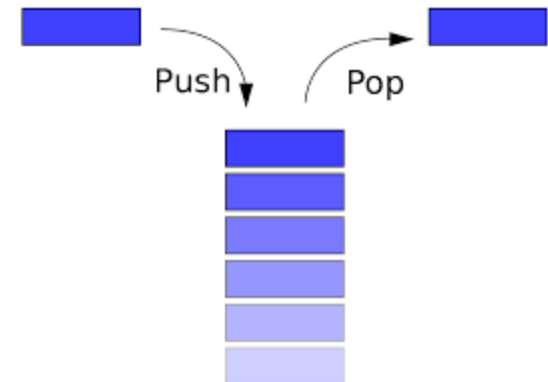
// Wenn kein Element vorhanden ist, wird eine NoSuchElementException geworfen.

```
public T pop() throws NoSuchElementException
```

// true, wenn keine Elemente enthalten sind, sonst false .

```
public boolean isEmpty()
```

Schreibe einen Unit Test die Klasse.



MyStack<T>: Lösung

```
public class MyStack<T> {  
  
    private final List<T> elements = new LinkedList<>();  
  
    public void push(T element) {  
        elements.addLast(element); // oder einfach add  
    }  
  
    public T pop() throws NoSuchElementException { //  
        if(isEmpty()) {  
            throw new NoSuchElementException("Stack is empty");  
        }  
        return elements.removeLast();  
    }  
  
    public boolean isEmpty() {  
        return elements.isEmpty();  
    }  
}
```

```
class MyStackTest {  
  
    @Test  
    void testPushAndPop() {  
        // Arrange  
        MyStack<Integer> stack = new MyStack<>();  
  
        // Act  
        stack.push(1);  
        stack.push(2);  
        int result = stack.pop();  
  
        // Assert  
        assertEquals(2, result);  
        assertFalse(stack.isEmpty());  
    }  
  
    @Test  
    void testPopEmptyStack() {  
        // Arrange  
        MyStack<Integer> stack = new MyStack<>();  
  
        // Act & Assert  
        assertThrows(NoSuchElementException.class, stack::pop);  
    }  
  
    @Test  
    void testIsEmpty() {  
        // Arrange  
        MyStack<Integer> stack = new MyStack<>();  
  
        // Act & Assert  
        assertTrue(stack.isEmpty());  
        stack.push(1);  
        assertFalse(stack.isEmpty());  
    }  
}
```


Longest String

Finde in einer Collection den längsten String und gebe ihn zurück.
Falls kein String gefunden wird, gebe einen leeren String "" zurück.

Beispiel:

```
Collection<String> words = List.of("Apple", "Orange", "Strawberry", "Banana");  
String longestString = LongestString.findLongestString(words); // Strawberry
```

Überlege dir, welcher Edge-Case hier wahrscheinlich auftreten könnte.
Schreibe einen Unit Test für deine Lösung.

Longest String: Lösung

```
public static String findLongestString(Collection<String> words) {  
    String longestString = "";  
    for (var word : words) {  
        if (word.length() > longestString.length()) {  
            longestString = word;  
        }  
    }  
    return longestString;  
}
```

// Alternativen

```
public static String findLongestString(Collection<String> words) {  
    return words.stream().max(String::compareTo).orElse("");  
    return words.stream().max((w1, w2) -> w1.length() - w2.length()).orElse("");  
    return words.stream().max(Comparator.comparingInt(String::length)).orElse("");  
}
```

Longest String – Unit Test: Lösung

```
public class LongestStringTest {  
  
    @Test  
    public void testFindLongestString() {  
        // Arrange  
        var strings = List.of("Apple", "Orange", "Banana", "Strawberry");  
        // Act  
        String longestString = LongestString.findLongestStringLoop(strings);  
        // Assert  
        assertEquals("Strawberry", longestString);  
    }  
  
    @Test  
    public void testFindLongestStringWithEmptyCollection() {  
        // Arrange  
        List<String> strings = List.of();  
        // Act  
        String longestString = LongestString.findLongestStringLoop(strings);  
        // Assert  
        assertEquals("", longestString);  
    }  
}
```

```
@Test  
public void testFindLongestStringWithSingleElement() {  
    // Arrange  
    var strings = List.of("Apple");  
    // Act  
    String longestString = LongestString.findLongestStringLoop(strings);  
    // Assert  
    assertEquals("Apple", longestString);  
}  
  
@Test  
public void testFindLongestStringWithEqualLengthStrings() {  
    // Arrange  
    var strings = List.of("Cat", "Dog", "Fox"); // Edge Case  
    // Act  
    String longestString = LongestString.findLongestStringLoop(strings);  
    // Assert  
    assertEquals("Cat", longestString);  
}  
  
@Test  
public void testFindLongestStringNull() {  
    // Arrange  
    var strings = Arrays.asList("Cat", null, "Horse");  
  
    // Assert  
    assertThrows(NullPointerException.class, () -> LongestString.findLongestStringLoop(strings));  
}  
}
```

Linear Search

Implementiere eine Methode, die eine **Collection** von ganzen Zahlen sowie eine gesuchte Zahl entgegennimmt.

Die Methode gibt den **Index** der gesuchten Zahl in der **Collection** zurück, falls die Zahl vorhanden ist, sonst -1.

Bei mehrfachen Vorkommen der gesuchten Zahl, soll der **Index** der zuerst gefundenen Zahl zurückgegeben werden.

Tipp: Es könnte hilfreich sein, die empfangene Collection in eine Liste umzuwandeln.

Beispiel:

```
Collection<Integer> numbers = List.of(1, 2, 3, 4, 5);  
int index = LinearSearch.findIndexOfNumber(numbers, 3); // 2
```

Schreibe einen Unit Test für deine Lösung.

Linear Search: Lösung

```
public static int findIndexOfNumber(Collection<Integer> numbers, int numberToSearchFor) {  
    return new ArrayList(numbers).indexOf(numberToSearchFor);  
}
```

```
var numbersAsList = new ArrayList<>(numbers);  
for (int index = 0; index < numbersAsList.size(); index++) {  
    if (numbersAsList.get(index) == numberToSearchFor) {  
        return index;  
    }  
}  
return -1;  
}
```

```
public static int findIndexOfNumberStream(Collection<Integer> numbers, int numberToSearchFor) {  
    var numbersAsList = new ArrayList<>(numbers);  
    return IntStream.range(0, numbersAsList.size())  
        .filter(index -> numbersAsList.get(index) == numberToSearchFor)  
        .findFirst()  
        .orElse(-1);  
}
```

Linear Search – Unit Test: Lösung

```
public class LinearSearchTest {
```

```
    @Test
```

```
    public void testFindIndexOfNumberPresent() {  
        // Arrange  
        var numbers = List.of(1, 2, 3, 4, 5);  
        // Act  
        int index = LinearSearch.findIndexOfNumber(numbers, 3);  
        // Assert  
        assertEquals(2, index);  
    }
```

```
    @Test
```

```
    public void testFindIndexOfNumberNotPresent() {  
        // Arrange  
        var numbers = List.of(1, 2, 3, 4, 5);  
        // Act  
        int index = LinearSearch.findIndexOfNumber(numbers, 6);  
        // Assert  
        assertEquals(-1, index);  
    }
```

```
    @Test
```

```
    public void testFindIndexOfNumberEmptyCollection() {  
        // Arrange  
        List<Integer> numbers = List.of();  
        // Act  
        int index = LinearSearch.findIndexOfNumber(numbers, 1);  
        // Assert  
        assertEquals(-1, index);  
    }
```

```
    @Test
```

```
    public void testFindIndexOfNumberMultipleOccurrences() {  
        // Arrange  
        var numbers = List.of(1, 3, 3, 4, 5);  
        // Act  
        int index = LinearSearch.findIndexOfNumber(numbers, 3);  
        // Assert  
        assertEquals(1, index);  
    }
```

```
    @Test
```

```
    public void testFindIndexOfNumberNull() {  
        // Arrange  
        var numbers = Arrays.asList(1, null, 3);  
  
        // Act  
        int index = LinearSearch.findIndexOfNumber(numbers, 3);  
  
        // Assert  
        assertEquals(2, index);  
    }  
}
```

Find MinMax

Implementiere eine Methode, die eine Collection von Ganzzahlen entgegennimmt und ein **record** MinMax mit zwei Attributen zurückgibt: dem kleinsten und dem größten **Wert** in der Eingabe-Collection.

Falls nur ein Element vorhanden ist, ist dieses Element zugleich Minimum und Maximum.

Wenn keine gültigen Werte gefunden werden, wird eine **NoSuchElementException** geworfen.

Beispiel:

```
Collection<Integer> integers = List.of(1, 2, 3, 4, 5, 0);  
MinMax minMax = MinMaxUtil.findMinMax(integers); // MinMax[min: 0, max: 5]
```

Schreibe einen Unit Test für deine Lösung.

Find MinMax: Lösung

```
public static MinMax findMinMax(Collection<Integer> numbers){
    int min = numbers.stream().mapToInt(Integer::intValue).min().getAsInt(); // wirft eine NoSuchElementException, falls Element nicht vorhanden
    int max = numbers.stream().mapToInt(Integer::intValue).max().getAsInt();

    // int min = numbers.stream().min(Integer::compare).orElseThrow(NoSuchElementException::new);
    // int max = numbers.stream().max(Integer::compare).orElseThrow(NoSuchElementException::new);

    // int min = numbers.stream().mapToInt(i->i).min().orElseThrow(NoSuchElementException::new);
    // int max = numbers.stream().mapToInt(i->i).max().orElseThrow(NoSuchElementException::new);

    return new MinMax(min, max);
}

public static MinMax findMinMax(List<Integer> numbers) {
    if(numbers.isEmpty()) {
        throw new NoSuchElementException("List is empty");
    }

    int min = Integer.MAX_VALUE;
    int max = Integer.MIN_VALUE;
    for (int number : numbers) {
        if (number < min) {
            min = number;
        }
        if (number > max) {
            max = number;
        }
    }
    return new MinMax(min, max);
}
```


Find MinMax: Lösung

```
public class MinMaxUtilTest {
```

```
    @Test
    void testFindMinMax() {
        // Arrange
        List<Integer> integers = List.of(1, 2, 3, 4, 5);
        // Act
        MinMaxUtil.MinMax minMax = MinMaxUtil.findMinMax(integers);
        // Assert
        assertEquals(1, minMax.min());
        assertEquals(5, minMax.max());
    }
```

```
    @Test
    void testFindMinMaxWithNegativeNumbers() {
        // Arrange
        List<Integer> integers = List.of(-1, -2, -3, -4, -5);
        // Act
        MinMaxUtil.MinMax minMax = MinMaxUtil.findMinMax(integers);
        // Assert
        assertEquals(-5, minMax.min());
        assertEquals(-1, minMax.max());
    }
```

```
    @Test
    void testFindMinMaxWithDuplicates() {
        // Arrange
        List<Integer> integers = List.of(1, 2, 3, 5, 1, 3, 4, 5);
        // Act
        MinMaxUtil.MinMax minMax = MinMaxUtil.findMinMax(integers);

        // Assert
        assertEquals(1, minMax.min());
        assertEquals(5, minMax.max());
    }
```

```
    @Test
    void testFindMinMaxOneNumberOnly() {
        // Arrange
        List<Integer> integers = List.of(1);
        // Act
        MinMaxUtil.MinMax minMax = MinMaxUtil.findMinMax(integers);
        // Assert
        assertEquals(1, minMax.min());
        assertEquals(1, minMax.max());
    }
```

```
    @Test
    void testFindMinMaxAllIdenticalNumbers() {
        // Arrange
        List<Integer> integers = List.of(1, 1, 1);
        // Act
        MinMaxUtil.MinMax minMax = MinMaxUtil.findMinMax(integers);
        // Assert
        assertEquals(1, minMax.min());
        assertEquals(1, minMax.max());
    }
```

```
    @Test
    void emptyList() {
        assertThrows(NoSuchElementException.class, () -> MinMaxUtil.findMinMax(List.of()));
    }
}
```